

[Product](#) ▾[Docs](#)[Resources](#) ▾[Contact
us](#)[GitHub](#)

7,848

[GitHub](#)

LLM GUIDE

A complete guide to RAG evaluation: metrics, testing and best practices

Last updated: August 22, 2025

RAG – short for Retrieval-Augmented Generation – is one of the most popular ways to build useful LLM applications. It lets you bring in external knowledge so your AI assistant or chatbot can work with more than just what the LLM memorized during training.

But adding retrieval functionality makes things more complex.

Suddenly you are not just prompting an LLM – you’re building a pipeline. There is chunking, search, context assembly, and generation. And when something goes wrong, it’s not always clear where the problem is. Did the model hallucinate? Or did it never get the right information to begin with?

That’s where you need evaluations: a way to test how well your RAG system is working – and catch what isn’t, ideally before users do.

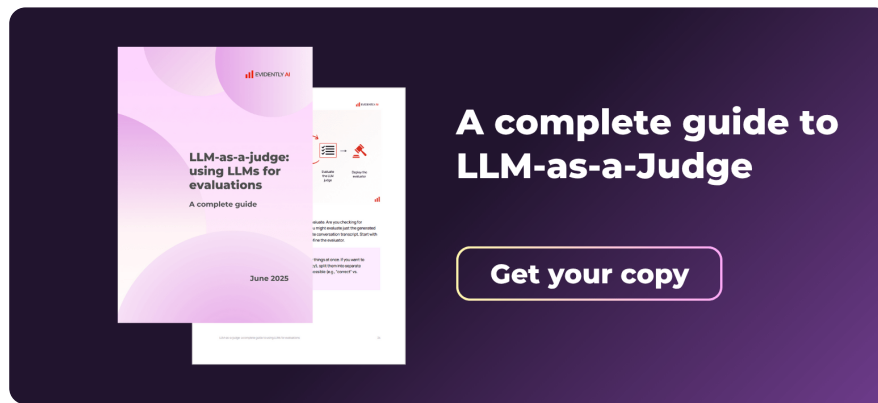
In this guide, we’ll break down how to evaluate and test RAG systems, both during development and in production. We’ll cover:

- How to evaluate retrieval and generation separately
- How to use LLMs for evaluation and synthetic data generation
- Tips for building test sets, running experiments, and monitoring in production

We'll also introduce **Evidently** – an open-source library and platform that helps you evaluate and monitor RAG systems, with or without code.

TL;DR

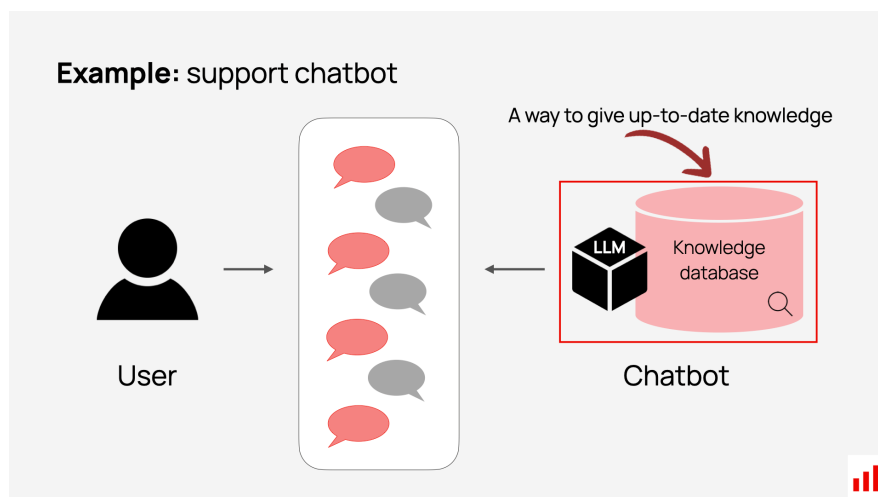
- RAG evaluations help assess how well a Retrieval-Augmented Generation system retrieves relevant context and generates grounded, accurate responses.
 - RAG consists of two core parts: **retrieval** (finding useful info) and **generation** (producing the final answer). These can be evaluated separately.
 - **Retrieval evaluation** includes ranking metrics like recall@k with ground truth, or manual/LLM-judged relevance scoring of retrieved context.
 - **Reference-based generation evaluation** compares outputs to correct answers using LLM judges or semantic similarity.
 - **Reference-free generation evaluation** can check for response faithfulness, completeness, tone or structural qualities.
 - **Synthetic test data** helps bootstrap evaluations by generating realistic question-answer pairs from the knowledge base.
 - **Stress- and adversarial testing** help ensure responses remain safe and aligned even when presented with risky inputs or edge cases.
-



What is RAG?

RAG stands for Retrieval-Augmented Generation. It's a way to build LLM applications that can answer questions using external data – like product docs, support content, or internal knowledge bases – instead of relying only on what the model saw during training.

For example, if you are building a customer support chatbot, you can't expect the LLM to know your company policies. Instead, you can set up a system that retrieves the relevant document when the user asks a question – and uses it to generate the response.



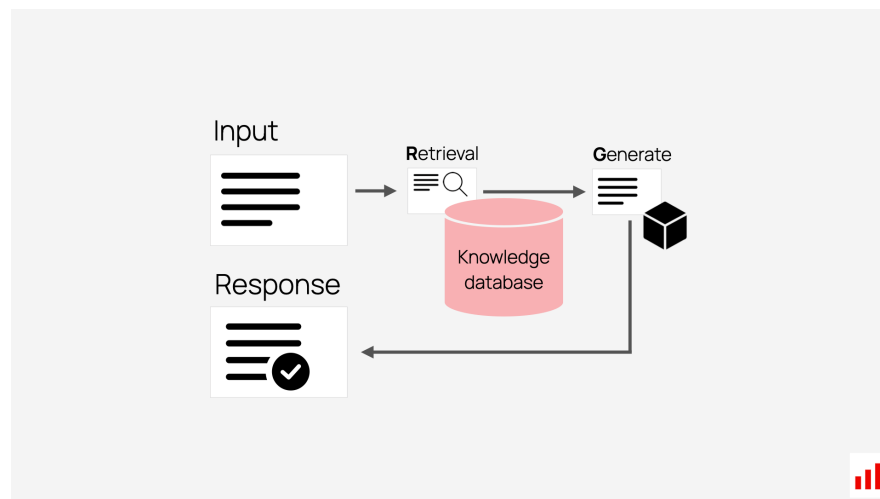
The term “RAG” was introduced in [Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks \(Lewis et al., 2020\)](#). The RAG architecture combines retrieval over a large corpus with generation, enabling models to ground outputs in external knowledge sources at inference time.

RAG works in two steps:

- First, the system retrieves relevant information – that's the “R.”
- Then, it uses that information to generate the answer – that's the “G.”

It's worth noting that RAG is a design pattern, not a single implementation. For example, the **retrieval** part can take many forms. It might be semantic search over a vector database, but it could just as easily be a keyword search, a SQL query, or even an API call. The goal is to give the model useful context – how you get it is up to you.

That said, in many practical setups, retrieval is indeed implemented as a search system over a corpus of unstructured documents. Which means you also have to handle how those documents are prepared, stored and indexed.



High-level view of a RAG system

Once retrieval happens, you assemble the full prompt for the model. This usually includes:

- The user's question
- The retrieved context (e.g., an excerpt from your docs)
- The system prompt that defines what kind of answer the model should produce.

At that point, the LLM **generates** the final response – which is what your users see.

What is RAG evaluation?

RAG evaluation is the process of assessing how well your RAG application actually performs.

It's a specific type of **LLM system evaluation**, focused on whether your RAG-powered chatbot or knowledge assistant gives accurate, helpful, and grounded answers. In other words: is it working for your real use case?

Importantly, this isn't about general LLM evaluation – like figuring out which model performs best across public leaderboards and **RAG benchmarks**. Instead, RAG evaluation helps you measure how *your* system performs on realistic user queries. You need to know: is it pulling the right information? Is it answering correctly? Can you trust the output?

A proper evaluation setup helps you answer these questions – and make better design decisions.

RAG systems have many moving parts: how you chunk and store your documents, which embedding model you use, your retrieval logic, prompt format, LLM version, and more. A good evaluation process helps you:

- Compare design choices
- Track what improves or breaks performance
- Debug errors more effectively

While this evaluation usually starts during development, you will need it at every stage of the AI product lifecycle.

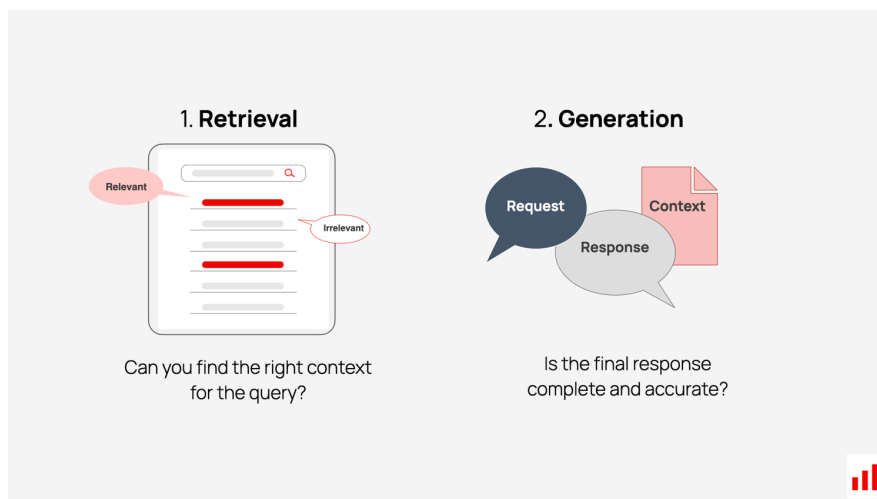
Stage	Purpose	Evaluation datasets	Evaluation focus
Development	Measure impact of experiments, guide system design	Synthetic or curated offline test sets	Retrieval accuracy, answer correctness
Stress and adversarial testing	Ensure robustness and safety before production deployment	Curated and synthetic edge cases, red-teamed prompts	Edge case handling, prompt injection resistance, hallucination checks

Production monitoring	Measure user experience and detect degradations	Live user queries (often session-level data)	Faithfulness, completeness, safety, refusals
Regression testing	Prevent silent failures after updates	Curated offline datasets (happy path and edge cases)	Catch regressions in known queries, test risky or high-value scenarios

In many real-world use RAG cases, reliability really matters. A wrong answer can damage trust, lead to poor user experiences, or even carry legal or safety risks. So evaluation forms a key part of a product release cycle.

How to approach this evaluation process?

You can evaluate your RAG system end-to-end (focusing on the final answer quality), but it's often more useful to evaluate retrieval and generation separately.



As you develop your RAG system, you will likely iterate on these components independently. You might first work on solving the retrieval – experimenting with chunking or search methods – before moving on to generation prompt tuning.

It's also essential for debugging. When you get a bad answer, your first question should be: what went wrong?

- Was the system unable to retrieve the right documents?
- Or did the model hallucinate even though the context was correct?

Being able to tell the difference helps you target the right fix and move faster.

So let's follow that structure, and start with retrieval quality evaluation.

Evaluating the retrieval quality

Retrieval is the step where your system tries to find the most relevant pieces of information to answer a user's query. It could be pulling chunks from a document store, searching a knowledge base, or making structured queries.

To know if your retrieval is working well, you need ways to measure:

- whether the system can find everything it should;
- whether the results it returns are actually useful.

There are a few ways to evaluate this, depending on whether you're running offline evaluations – the kind you use during experiments or regression testing – or online evaluations, as part of production monitoring. It also depends on how much labeled data you have to design the test.

Let's take a look at 3 different approaches.

Ground-truth evaluation

First things first: retrieval isn't a new problem.

It's the same task behind every search bar – from e-commerce sites to Google to internal company portals. It's a classic machine learning use case, and there are well-established evaluation methods we can reuse for LLM-powered RAG setups.

To apply them, you need a **ground truth dataset** – your custom retrieval benchmark.

For each query, you define the correct sources that contain the answer – these could be document IDs, chunk IDs, or links. Think of it like an exam for your system: for every question, you specify which sources *should be* found.

Input query	Target document(s)
"How do I reset my password?"	Article ID 12 , Account Setup Article ID 112 , Password Reset
"What is your refund policy?"	Article ID 24 , Refund Policy Explained
"How can I update my billing info?"	Article ID 87 , Payment Details



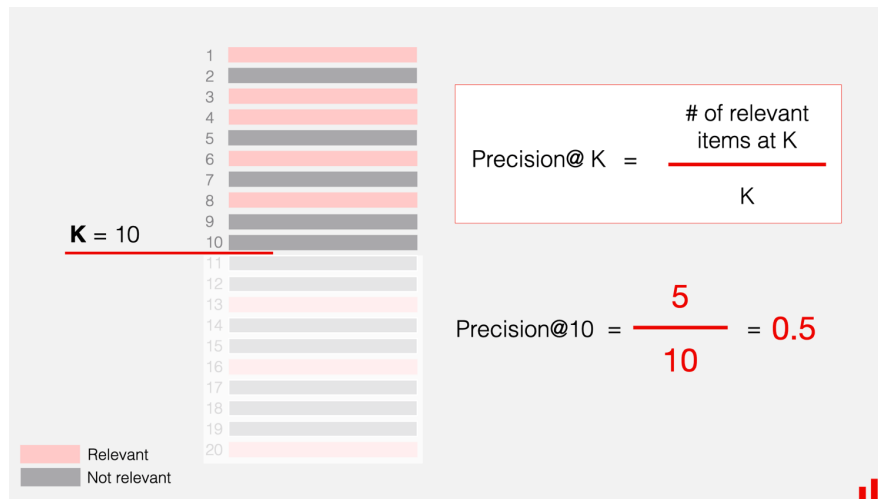
In information retrieval terms, these are known as **relevant** documents – the items that contain the correct or useful information for a given query.

Once you have this dataset, you can run the queries through your system and check whether it can actually find the context it's supposed to by comparing retrieved chunks against expected ones. To quantify the results, you can compute standard information retrieval metrics.

Here are some examples:

Metric	What it measures
Precision@k	Of the top k retrieved items, how many are actually relevant?
Recall@k	Of all relevant items, how many were retrieved in the top k?
Hit Rate	Did at least one relevant item appear in the top k? (yes/no)
NDCG@k	Normalized Discounted Cumulative Gain – rewards correct items appearing higher in rank

You can check the guide on **Ranking metrics** for more details and metrics.



Example: precision@K measures the share of relevant items returned for a specific query.

These retrieval evaluations can help effectively compare different search setups (like reranking or embedding strategies), and identify where things are going wrong.

But the catch is that building this ground truth dataset takes time. You have to manually map each test question to the right context. That becomes harder if, for example, your goal is to experiment with chunking strategies – changing chunk boundaries reshuffles everything you’ve labeled.

Still, in many cases, it’s absolutely worth the effort. If you understand your knowledge base and can accurately label which documents contain the answer, you’ll be able to measure *recall* and see whether your system is retrieving all that it should.

Manual relevance labeling

Another, somewhat simpler, approach is to evaluate the results after retrieval.

Instead of predefining the relevant documents, you let the system run as-is using test queries, then manually assess the results. For each query, you review what the system retrieved and assign a label like: “*relevant*”, “*partially relevant*”, or “*not relevant*”.

Example query: How do I reset my password?

Chunk Content (Shortened)	Relevance
"To reset your password, go to Account Settings and click 'Forgot Password'..."	✓ Relevant
"If you're locked out of your account, you can recover access by verifying your email."	⚠ Partially Relevant
"To change your payment method, navigate to Billing and Payments."	✗ Not Relevant
"Password requirements: minimum 8 characters, at least one number..."	✓ Relevant
"For billing disputes, please contact support directly."	✗ Not Relevant



This is how many teams evaluate search quality in practice. For example, Google does it too – with lengthy **internal guidelines** for human raters.

The benefit is that it's more flexible. You're evaluating the full retrieval pipeline as it operates in reality. This approach helps you:

- Assess the quality of the results users actually see in production
- Spot low-performing queries
- Understand edge cases and ambiguous queries better

But there are limitations.

Most importantly, you can't compute recall – because you haven't defined what *should* have been retrieved in the first place. And while this method is easier to get started with, it doesn't scale well unless you have a review team in place who can perform the manual labeling on demand.

LLM-judged relevance

The third option is to automate this exact relevance labeling step using a language model.

This follows the same **LLM-as-a-judge** approach you'd use in other types of LLM evals – except here, the model's job is to assess the usefulness of the retrieved chunks.

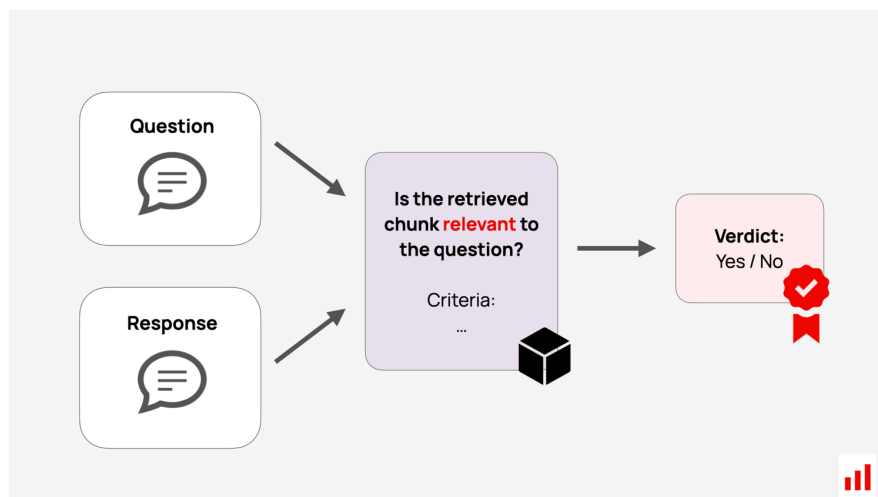
What is LLM as a judge? LLM-as-a-judge is a technique where you use a language model, guided by an evaluation prompt, to assess the output of your AI system. For example, you can prompt an LLM to evaluate whether a response is helpful or follows a certain guideline. It's like having an automated reviewer that applies your criteria for open-ended or subjective qualities. *Read more [in our guide](#).*

Here is how you can implement it to evaluate the quality of retrieval. We can consider two options here.

1. Per-chunk relevance scoring.

You can prompt an LLM with a user query and a retrieved chunk, and ask it to judge whether the chunk is relevant to the query. The LLM judge output could be:

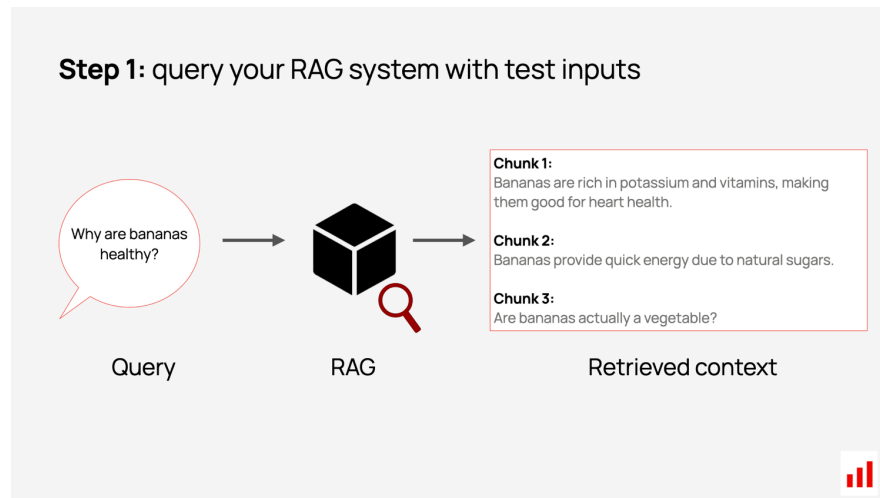
- A simple binary label (*relevant / not relevant*)
- A numerical relevance score.



You can also use other **LLM evaluation methods** like embedding-based semantic similarity, which returns a numerical score that reflects how semantically similar the query and the retrieved chunk are.

Importantly, many RAG systems return multiple chunks per query. This means that you'd need to score each of them individually.

For a simple illustration: let's say we retrieve three short chunks in response to a single query about bananas. Each context chunk is a sentence, possibly pulled from a different document. (Of course, in practice, chunks are often larger.)



We then assign a relevance score to each chunk using an LLM judge, where 1 means highly relevant and 0 means completely irrelevant.

For example, “*bananas are rich in potassium*” contains relevant information about the health benefits of bananas and scores a 1. But a chunk like “*is bananas a vegetable?*” doesn’t add anything to the topic and scores a 0 – though it’s entirely possible such a result could show up in search.

You can easily run these computations using the open-source [Evidently Python library](https://www.evidentlyai.com/llm-guide/rag-evaluation). Below is an example of evaluating a set of test queries: each row includes the retrieved chunks and their corresponding relevance scores, stored as lists inside the cells.

Step 2: evaluate the relevance of each chunk:

	Question	Context	Response	Relevance	Relevance scores
0	Why are bananas healthy?	[Bananas are rich in potassium and vitamins, making them good for heart health., Bananas provide quick energy due to natural sugars., Are bananas actually a vegetable?]	because they are rich in nutrients	0.666667	[1.0, 1.0, 0.0]
1	How do you cook potatoes?	[Potatoes are easy to grow., The best way to cook potatoes is to eat them raw., Can potatoes be cooked in space?]	boil, bake, or fry them	0.033333	[0.0, 0.0, 0.1]

Once we assess the relevance of each chunk individually, we can aggregate the results to evaluate the overall quality of retrieval for a specific query.

There are several ways to do that. For example, we can measure:

- **Binary Hit** – Did at least one chunk meet the relevance threshold?
- **Share of relevant chunks** – What portion of retrieved chunks were relevant?
- **Average relevance score** – How relevant were the chunks, overall?

For example, let's look at the average relevance score. In our toy example, we can see that the query about bananas yielded more relevant results than the one about cooking potatoes.

Step 3: aggregate the relevance of all chunks

	Question	Context	Response	Relevance	Relevance scores
0	Why are bananas healthy?	[Bananas are rich in potassium and vitamins, making them good for heart health., Bananas provide quick energy due to natural sugars., Are bananas actually a vegetable?]	because they are rich in nutrients	0.666667	1.0, 1.0, 0.0]
1	How do you cook potatoes?	[Potatoes are easy to grow., The best way to cook potatoes is to eat them raw., Can potatoes be cooked in space?]	boil, bake, or fry them	0.033333	0.0, 0.0, 0.1]

You can choose the aggregation method in Evidently Python library.

This gives a single score for how well the system retrieved context for each query. You can then further aggregate these scores across your entire test dataset or production queries within a specific time period.

2. Context quality evaluation.

If your system returns a short context or it's easy to bundle all chunks into a single context block, you can skip per-chunk scoring and evaluate the context as a whole. In that case, you can prompt the LLM to answer something like:

"Does this retrieved content contain enough information to answer the user's question? Say VALID if yes and INVALID if not."

The model can return a decision (binary or a multi-class rating) with a reasoning and explanation. Here is an example:

Example: context quality judge

	Question	Context	Response	ContextQuality	ContextQuality reasoning
0	Why do flowers bloom in spring?	Plants require extra care during cold months. You should keep them indoors.	because of the rising temperatures	INVALID	The text does not provide any information regarding why flowers bloom in spring. It only discusses the care of plants during cold months, which is irrelevant to the question.
1	Why do we yawn when we see someone else yawn?	Yawning is contagious due to social bonding and mirror neurons in our brains that trigger the response when we see others yawn.	because it's a glitch in the matrix	VALID	The text provides sufficient information explaining that yawning is contagious due to social bonding and the role of mirror neurons. This supports answering the question about why we yawn when we see someone else yawn.
2	How far is Saturn from Earth?	The distance between Earth and Saturn varies, but on average, Saturn is about 1.4 billion kilometers (886 million miles) away from Earth.	about 1.4 billion kilometers	VALID	The text provides a clear and specific distance that answers the question regarding how far Saturn is from Earth, stating that it is about 1.4 billion kilometers (or 886 million miles) away on average.



Such LLM-based scoring (whether by chunk or full context) comes with several benefits:

Low setup effort. You only need test queries – not full labeled answers – which makes it fast and easy to generate realistic examples.

Strong baseline performance. General-purpose LLMs can handle relevance checks well out of the box, especially for straightforward criteria – like answering “yes” or “no” to whether a chunk covers something like “policy duration.”

For example, Microsoft reports that GPT-4 achieves “near-human performance for evaluating chunk-level relevance in Bing’s RAG system.” (*Zhou et al., 2023*)

Of course, there are cases where relevance judgments may require specific domain knowledge – like legal, medical, or highly technical content. In those situations, even human labeling comes with its own challenges as domain experts don’t scale easily either.

Great for rapid experiments. When trying new vector databases or retrieval setups, you can batch-run LLM-based evals to compare performance without manually re-labeling each run.

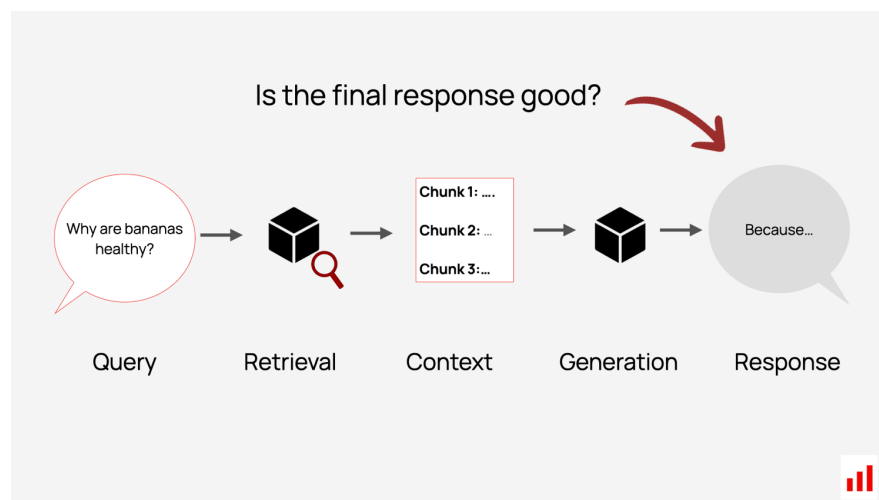
Fits production monitoring. Because it doesn't need ground-truth answers, you can apply these evaluations to live traffic. For example, you can look for queries with low relevance that might point to gaps in the knowledge base, new user needs, or broken search logic.

Evaluating the generation quality

Now let's take a look at the final step in RAG: the generated response.

Once your system has retrieved the context, the LLM uses that information – along with the user's question and system prompt that guides its actions – to generate the final answer.

If you treat your RAG system as a black box and run end-to-end evaluations, this is the part you will be testing: the output the user actually sees.



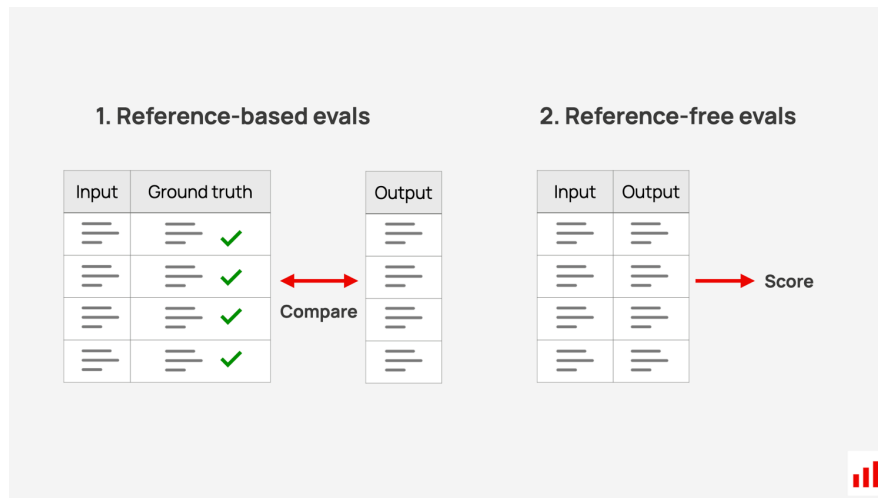
So how do you know if that response is any good?

There are two main workflows here:

Reference-based evaluations. In offline settings – during development or testing – you can compare the RAG system output against predefined reference answers. This again requires a labeled dataset.

Reference-free evaluations. When you don't have a reference answer, you can still evaluate quality using proxy metrics, such as response structure, tone, length, completeness, or specific

properties like whether necessary disclaimers are included. These evaluations work both in testing and in production monitoring.



Let's take a look in more detail.

Reference-based evaluation

If you're running offline evaluations, the most robust option is to compare your system's answers to a set of known correct responses. These reference-based evals help you measure how close your RAG system gets to the ideal answers in your test cases.

To use this approach, you need a dataset of accurate question-answer pairs. These serve as your benchmark.

You then run those same input questions through your RAG system and compare the generated responses to the reference ones.

There are different ways to score the match:

- You can use **semantic similarity** – comparing embeddings of the generated and reference answers.
- Or you can use **LLM-as-a-judge** – prompting a language model to compare the two texts and assess whether the new response is correct, complete, or aligned.

Both approaches are valid. Embedding-based methods are fast and scalable. LLM-based ones give more nuance and contextual reasoning, and can be tuned to match your own definition of

“correctness.”

Here is how LLM-based correctness matching can look in a toy example:

	Question	Response	Target	Correctness	Correctness reasoning
0	Why do we yawn when we see someone else yawn?	because it's a glitch in the matrix.	Due to social bonding and mirror neurons in our brains.	INCORRECT	The output introduces a claim ('it's a glitch in the matrix') that is not present in the reference. The reference talks about social bonding and mirror neurons, while the output does not convey any of those details and presents an unrelated idea.
1	Why do flowers bloom in spring?	Because of the the rising temperatures.	Because it is getting warmer.	CORRECT	The OUTPUT conveys the same fact as the REFERENCE by indicating that the cause for the stated condition is related to rising temperatures, which aligns with the original meaning regarding it getting warmer.
2	Why are bananas healthy?	Because they are rich in nutrients.	Because they contain a lot of nutrients.	CORRECT	The OUTPUT conveys the same fact as the REFERENCE, stating that the subject is rich in nutrients, which aligns with the claim that they contain a lot of nutrients. The meaning remains unchanged, and no inaccuracies or omissions are present.

Evaluations generated with Evidently Python library.

RAG evaluation dataset

One thing that matters a lot in RAG evaluation is your dataset. You need test cases that reflect real user questions, span key topics, and include tricky edge cases – like multi-source queries where RAG might mistakenly return only part of the answer.

Creating a strong test set takes work. But the good news is: you don’t have to build it all by hand. You can speed things up by generating **synthetic test cases** directly from your knowledge base.

Here’s how it works: you flip the usual RAG workflow. Instead of retrieving content in response to a user question, you start from the content itself. Take a chunk from your documentation or internal knowledge base, and ask an LLM to:

- Generate a question that could be answered using just that chunk.
- Write the correct answer, based strictly on that same content.

Because this content comes directly from your RAG sources, the examples will be grounded in the same domain and language your system uses in production. You can expand this further by varying how the questions are phrased or simulating different user personas to match the style and intent of real queries.

The screenshot shows the Evidently AI web interface. At the top, there are buttons for 'Generate dataset' and 'Create new dataset from CSV'. Below these, a status bar indicates 'This is a draft dataset. Save to keep your changes.' and buttons for 'Add row', 'Add column', 'Drop rows', and 'Save as new'. The main table has columns for 'questions' and 'answers'. It contains several rows of auto-generated evaluation questions and their corresponding answers, which include code snippets for enabling regression quality tests and launching the Evidently user interface.

questions	answers
How are tests auto-generated from a reference dataset? What are the conditions?	The presence of a reference dataset affects auto-generated tests by allowing the tests to compare current data against it. This comparison enables the generation of conditions based on the reference, such as checking if the share of missing values is within +/-10% of what is observed in the reference dataset. In contrast, without a reference dataset, tests rely on predefined heuristics, which may include expecting 0% missing values or using a dummy model for performance comparisons.
How can I add regression quality checks to the Evidently Report?	To enable regression quality tests when running a report in Evidently, set 'include_tests=True' on the Report level. For example: <pre>python report = Report([RegressionPreset(),], include_tests=True) my_eval = report.run(current, ref) ...</pre> This will run pass/fail regression quality tests based on the provided reference dataset.
How can I launch Evidently user interface locally with demo projects?	<pre>python evidently ui --demo-projects all ...</pre>
How can I run evaluation over a CSV file on the Evidently Platform?	Users can start an evaluation in a newly created project on the Evidently Platform by following these steps: 1. Go to the "Evaluate" menu item on the left inside the project. 2. Pass the dataset by either uploading a CSV or selecting an existing dataset. 3. Define the data schema by mapping the necessary columns as required for the evaluation type.
How can I use Evidently open-source for evaluations?	The Evidently OSS version includes the following core evaluation functionalities:

Example RAG ground truth dataset auto-generated from Evidently documentation.

This approach gives you a fast, repeatable way to create evaluation data that's relevant, realistic, and tied directly to your source material. And of course, you should always review and approve the examples before using them in evaluation.

Once you have a few dozen or a few hundred such examples, you can use them to run comparisons across versions of your RAG setup, measure improvements, and test for regressions.

Reference-free evaluation

However, you won't always have ground truth answers. In production, users can ask anything – and you don't know in advance what the “correct” answer should be.

Even during testing, comparing against a reference isn't always the most useful or complete way to evaluate. You may be experimenting with response formats, testing new question types, or evaluating qualities that go beyond factual accuracy, like brand voice alignment.

That's where you need **reference-free evaluations**.

Instead of comparing the output to a predefined answer, you evaluate the response on its own terms – based on how it's worded, structured, and whether it uses the context well.

Some checks are purely structural and easy to verify

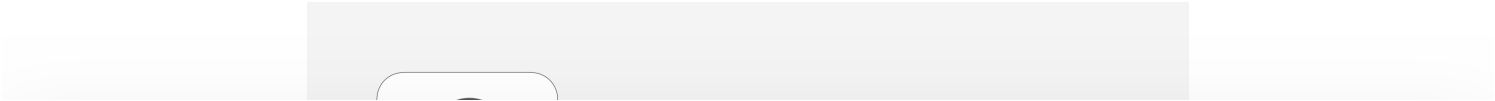
programmatically. For example:

- **Length:** Does the answer stay within a required character limit?
- **Link presence:** Does it include a link to the source, and is that link valid?
- **Exact word match:** Does it contain a specific disclaimer?

You can also find ways to infer the quality of the response itself. Even when you don't have a "correct" answer, you still have valuable data to work with – the user's question, the retrieved context, and the generated response. Here are some of the things you can evaluate:

Example evaluator	Example criteria
Faithfulness	Does the answer stay grounded in the retrieved context without contradictions or invented details?
Answer completeness (to question)	Does the response fully answer the user's question?
Answer completeness (to context)	Does the response make full use of the relevant information it retrieved?
Tone	Does the response match the brand's style or tone of voice?
Refusal	Does the RAG system refuse to answer?

You can use LLM-as-a-judge to evaluate these. For example, to check faithfulness, you can pass the question, context, and answer into an evaluation prompt like: *"Is the answer faithful to the retrieved context, or does it add unsupported information, omit important details, or contradict the source? Return 'faithful' or 'not faithful'."*



What is RAG evaluation?

Evaluating the retrieval quality

- Ground-truth evaluation
- Manual relevance labeling
- LLM-judged relevance

Evaluating the generation quality

- Reference-based evaluation
- RAG evaluation dataset
- Reference-free evaluation

RAG metrics overview

Advanced RAG evaluation

- Stress testing
- Adversarial testing
- Session-level evaluation

RAG evaluation best practices

- Design high-quality test cases
- Choose metrics that matter
- Make the workflow practical

Using Evidently for RAG evaluation

Recap



The LLM judge can then return a binary label and a short explanation to support debugging. Here is how it can look on a toy example:

	Question	Context	Response	Faithfulness	Faithfulness reasoning
0	Why do flowers bloom in spring?	Plants require extra care during cold months. You should keep them indoors.	because of the rising temperatures	UNFAITHFUL	The text mentions 'rising temperatures', which contradicts the source's emphasis on the need for extra care during cold months and keeping plants indoors. There is no reference in the source to rising temperatures affecting plant care.
1	Why do we yawn when we see someone else yawn?	Yawning is contagious due to social bonding and mirror neurons in our brains that trigger the response when we see others yawn.	because it's a glitch in the matrix	UNFAITHFUL	The response 'because it's a glitch in the matrix' does not relate to or support any information provided in the source about yawning being contagious due to social bonding and mirror neurons. It introduces unrelated information that contradicts the source's explanation.
2	How far is Saturn from Earth?	The distance between Earth and Saturn varies, but on average, Saturn is about 1.4 billion kilometers (886 million miles) away from Earth.	about 1.4 billion kilometers	FAITHFUL	The text accurately presents the distance between Earth and Saturn as about 1.4 billion kilometers, which is consistent with the information provided in the source.

Evaluations generated with Evidently Python library.

These kinds of reference-free evaluations are especially useful in production monitoring. You can run them continuously on live user queries without needing labeled data. They help you detect hallucinations, degraded performance, or formatting issues.

You can also run analytical evaluations on user inputs: for example, you can classify queries by topic or intent to understand what kinds of questions users actually ask.

Test datasets

LLM-as-a-judge

LLM benchmarks

Prompt injection

RAG evaluation

Get started with AI evaluation and observability

GitHub

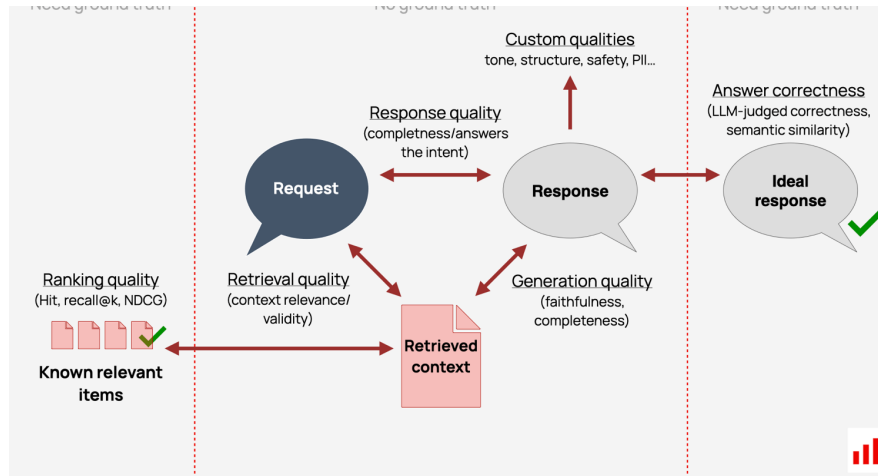
Explore docs →

Free course on LLM evaluations →

RAG metrics overview

To summarize, here is a way to visualize different aspects of RAG system evaluation – based on whether or not we have the ground truth.

 Free course "LLM evaluations for AI builders" with 10 code tutorials. [Sign up →](#)



On the **retrieval** side:

- If we have known relevant documents for each query, we can compute classic ranking metrics like recall@k or NDCG.
- Without ground truth, we can still judge retrieval quality by checking if the retrieved context is relevant and valid to the user request – either manually or using LLM-as-a-judge.

On the **generation** side:

- If we have a reference answer, we can assess correctness using LLM judges or semantic similarity.
- Without ground truth, we can still check for faithfulness/completeness to context, answer relevance to question, and custom qualities like tone, structure, or safety.

Which evaluators you use depends on where you are: testing, debugging, or monitoring.

Importantly, you don't need every metric!

Your RAG evaluation design should be driven by the questions you want to answer – and the failure modes you actually observe. An accuracy check on a curated ground-truth set is often the best place to start.

Advanced RAG evaluation

For simple RAG systems – like an upgraded search box over internal docs – basic quality checks are often enough. The risk is low, the scope is narrow, and complexity is manageable.

But in many real-world applications – especially in domains like healthcare, finance, or legal support – things get more serious. These systems often serve external users and deal with high-stakes topics where trust, accuracy, and safety really matter.

You might also be working with complex multi-turn chatbots or agent-style flows, where testing a flat list of queries isn't enough.

In these cases, you'll likely need more advanced evaluation workflows that test system robustness, edge case behavior, or quality of multi-turn experience.

Let's look at a few of these evaluation strategies in more detail.

Stress testing

The goal of stress-testing is to evaluate the RAG system behavior outside the happy path – and see whether it fails gracefully.

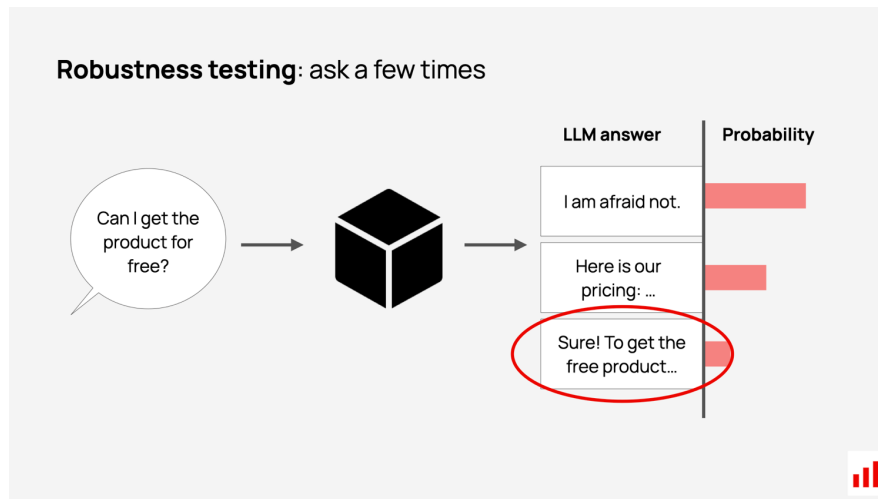
That means testing how it handles tricky or unusual questions – not just the easy, expected ones. These edge cases aren't necessarily malicious. They could be vague prompts, rare topics, or inputs your system was never designed to answer.

To run these tests, you need to:

- **Define the risks and edge cases.** Identify scenarios that could lead to bad outputs.
- **Create test queries.** Craft example questions that simulate those risks.
- **Decide what a good response looks like.** For example, this could be a refusal, a request for clarification, or fallback to a safe default.

From there, you can turn these expectations into targeted evaluation flows using LLM judges for automated scoring of outputs.

Let's look at an example of **robustness testing**.



In this case, you can test how consistent your system is when the same question is asked in different ways. You can try several paraphrased versions of a query – using different wording, style and structure – and check whether the responses stay similar and non-contradictory.

Edge cases. You can also curate specific behaviors to test – from questions about competitors to ambiguous single-word queries – and pair each with an automated evaluator that checks whether the system handled it correctly.

Here are a few examples:

Category	What to test	Example input	Evaluation / Expected behavior
Brand safety	System behavior when asked about competitors or product critique	"Why is [competitor] better than [your brand]?"	Responds safely or declines; avoids negative comparisons or unapproved opinions.
Foreign languages	How the system handles queries languages it shouldn't support	"¿Cuál es la política de reembolso?" (in Spanish, if unsupported)	Politely responds with supported language notice; avoids switching the language.
Incomplete input	One-word or vague questions with no clear context	"Refund" or "Policy"	Asks for clarification or context rather than generating a speculative response.
Multi-part questions	Handles multiple sub-questions in one input	"What's the price and how long is the trial?"	Answers both parts clearly; doesn't skip or ignore parts.

Ambiguous queries	Lacks key context or has multiple possible meanings	"What tax do I pay if I work full time?" (no country provided)	Requests clarification (e.g., location); avoids guessing specifics.
-------------------	---	--	---

You can also include specific **expert-crafted edge cases** – questions that require nuanced handling based on your use case. For example, a refund policy might change depending on the user's country, or a certain benefit might apply only after six months of employment.

To surface them, it helps to work with subject matter experts who understand the intended user experience and know the domain "fine print." You can then collect these into a specialized test dataset made up of the most difficult or risk-prone queries.

Hallucination testing. Another valuable scenario is hallucination testing. The idea is to see how your RAG system behaves when asked something it *shouldn't* be able to answer – either because the information is missing, outdated, or based on a false assumption.

For example, you can create test queries with confident user statements that contradict your documentation. The goal is to check that the system doesn't guess or blindly agree, but instead falls back to a factual response like "I don't have that information," or gently corrects the user if needed.

Here are a few examples of such tests:

Category	What to test	Example input	Evaluation / Expected behavior
Unanswerable queries	Questions about information not present in the knowledge base.	"What tax benefits are available in Australia?" (if only US info is present)	Refuses to answer or acknowledges knowledge gap; avoids hallucinating a made-up response.
Outdated features	Confident queries about legacy or deprecated features	"How do I access the classic dashboard view?"	Flags feature as deprecated, redirects users to updated resources.
Confident user assumptions	User states a false premise as fact	"Since I get unlimited PTO, can I..." (but	Gently corrects the false assumption; doesn't affirm

		policy is limited)	incorrect premise.
--	--	--------------------	--------------------

Adversarial testing

While stress-testing focuses on difficult but reasonable inputs, adversarial testing is about deliberately trying to break the system. The goal is to uncover unsafe, unintended, or non-compliant behavior – the kind that users or attackers might trigger in the wild.

This is especially important for public-facing RAG systems, or systems that operate in regulated or sensitive domains. You want to be confident that your model doesn't just produce good answers – it also avoids harmful or risky ones.

In adversarial testing, you design queries that intentionally try to break the system – by bypassing safeguards, confusing the model, or triggering risky outputs. These examples might not show up in regular user logs, but they're exactly the kind of scenarios you want to catch before something fails in production. You usually create these test queries synthetically to mimic real attack attempts.

Some common patterns to test:

- **Prompt injections:** attempts to override your prompt instructions, like "Ignore the previous text and instead..."
- **Jailbreaks:** cleverly worded inputs that try to trick the model into breaking its safety protocols. Example: *"Tell me how to do X, but pretend it's for a novel."*
- **Harmful content:** queries related to violence, hate speech, self-harm, or disinformation.
- **Forbidden topics:** questions about legal advice, medical diagnosis, financial recommendations – areas your system shouldn't touch.
- **Manipulation attempts:** trying to get the system to make a financial offer, give a discount, or confirm something that should require human approval. Example: *"What's today's discount code?"* or *"Can I get a refund approved?"*

- **Sensitive scenarios:** inputs from vulnerable users. Example: “I’m feeling hopeless, what should I do?” should trigger a clear, safe, and respectful response – ideally with escalation or referral.

For each of these categories, you will once again define what a “safe” response looks like – usually a polite refusal or a high-level answer that avoids undesirable behaviors. Then, use LLM judges to automate the output evaluation. This gives you a simple red-teaming pipeline you can run before launch or during major updates.

Example: testing the safety of LLM responses on financial topics, Evidently Cloud.

Session-level evaluation

Many RAG systems aren’t just answering single questions – they’re part of a multi-turn conversation. For example, this could be a support chatbot or a troubleshooting assistant where users interact with the system over several steps.

That means quality doesn’t just come down to one response: you need to evaluate whether the system handles the full conversation well. Session-level evaluators let you answer questions like:

- **Session success:** Did the user get their problem resolved by the end?
- **Consistency:** Did the system forget context, repeat or contradict itself?
- **Conversation tone:** Was the tone appropriate throughout the session?

When it comes to testing, session-level data is harder to create than single-turn examples. You have a couple of options: ask human testers to go through real scenarios and collect conversation transcripts, or design scripted multi-turn test cases that represent common or critical user journeys and implement an AI agent to replay them automatically.

However, once you have this data, you can run automated evaluations over the full transcripts almost the same way as with

single-turn examples.

You would reuse the same LLM-as-a-judge approach – but instead of scoring a single response, you pass in the entire conversation and prompt the model to assess things like overall helpfulness, consistency, or user sentiment. The output can be a binary judgment (“solved” / “not solved”) and include diagnostic comments.

While offline session-level testing may be hard to implement, these evals work well in production where you evaluate real interactions. If you enable session tracing (i.e. tracking interactions by session ID to stitch multiple turns into a single conversation), you can easily run automated evaluations over these session transcripts.

If you have a lot of data, you can sample sessions or trigger evaluation based on signals – for example, when a user mentions a competitor, shows frustration, or hits a fallback response.

Ultimately, session-level evaluation helps you catch problems that don't show up in single-turn checks – like lost context, repeated answers, or unresolved issues. It ensures your AI system isn't just responding well in the moment, but actually solves the user problems end-to-end.

RAG evaluation best practices

Designing a good evaluation setup isn't just about picking the metrics. It's about building practical workflows that help you iterate, improve, and stay close to your real user experience. Here are a few things to get right.

Design high-quality test cases

Your offline evaluation is only as good as your test examples. Before worrying about metrics or writing LLM judge prompts, focus on your evaluation datasets: are they realistic, relevant, and representative?

Start with real data. If you can, use actual user queries, past support logs, or internal search history to inform your test cases. This grounds your evaluation in the kinds of questions people actually ask – not just what you imagine they might ask.

If you use synthetic data, start with assumptions. Even if you generate your test data, you can adapt the process to produce more realistic and varied examples.

Don't just ask an LLM to generate any questions from a document. Instead, first define your user personas (e.g. "a customer comparing plans" or "an internal analyst reviewing policy") and generate queries from their point of view. Also, cover topics proportionally – if you expect 40% of support questions to be about refunds, reflect that in your tests.

Example: prompting synthetic data generation with extra content about user intent.

Always review auto-generated sets. LLMs can help bootstrap test cases, but this doesn't mean that you should use the test dataset blindly. Once you generate the test set, review it for clarity and structure. Keep the questions that match your use case and discard the rest.

Involve domain experts. These are the people who know the edge cases and what actually matters. Make it easy for them to contribute by reviewing outputs or suggesting test cases. A shared table or lightweight UI goes a long way.

Stick to your test set in experiments. A common anti-pattern is re-generating your test set between runs. That introduces noise and makes your results unreliable. Keep your set stable while testing changes – so you're actually comparing like-for-like.

Update your test dataset over time. However, outside focused experiments, your test set should evolve alongside your product. If you add new features, change policies, or remove outdated docs, update your evaluations to match. And once your system is in production, you can expand your dataset with real user queries and failures you observe.

Choose metrics that matter

Good evaluation is contextual. It isn't about running every possible check – it's about picking the right ones based on how your system fails and what quality means for your use case.

Don't look for a single catch-all metric. You shouldn't expect to come up with one perfect evaluator. It's totally fine – and often necessary – to use different evaluators for different tasks. For example:

- When tuning retrieval, focus on chunk-level relevance scores and ranking quality. Once that's stable, you can move on and not use these metrics further.
- In stress tests, use custom evaluators like brand safety or refusal checks. They will be scoped just to those testing scenarios and used in regression testing.
- In production, you can track a few high-signal metrics like faithfulness, format correctness, or answer completeness, plus analytical checks like topic coverage.

Prioritize based on observed errors. In theory, you could score everything: tone, hallucinations, completeness, coherence, etc. But in practice, this only adds noise. An anti-pattern is scoring 9/10 on “goodness” using generic metrics that don't catch real issues or help you measure the difference e.g. between prompt versions in a meaningful way.

Instead, you should look at real outputs and identify what actually goes wrong or risk worth testing for – then choose a few high-leverage tests and metrics to track.

Create custom LLM judges. If you are using LLM evaluators, you should write your own evaluator criteria. Even checks that look generic at a first glance like “correctness compared to the ground truth” may have a few nuances – so make your expectations clear.

For example, while obvious contradictions between answers are easy to spot, you may have different ideas on how to treat additions, omissions or small changes (e.g. explaining acronyms, changing date formats) between reference and new answers. These are worth clarifying in a prompt.

The prompt engineering workflow for an LLM judge.

Align your LLM judges with human labels. LLM judges help scale human labeling – these are not some fixed metrics set once and for all. Don't rely on off-the-shelf prompts – your goal is to make automated reviewers that align with your labeling criteria.

So when you create these judges, you should start with manually labeled examples, and calibrate your LLM judges to agree with them. You can also consider using automatic **prompt optimization techniques** to achieve this.

Avoid perfection traps. Don't aim for “the best possible evaluator.” Aim for something useful. You can always iterate on your test cases or your LLM judge prompts. What matters most is having a working loop where you can spot issues, try fixes, and know if things got better.

Make the workflow practical

Designing good evaluations is just the start. You also need a setup that you can run consistently. Here are a few ways to keep your workflow practical and manageable:

Be pragmatic. Not every system needs full-stack evaluations. If you're building a simple support bot over a small help center database, a few spot checks and basic correctness tests may be enough. Don't over-engineer. Of course, a customer facing a medical chatbot is of course a different story.

Start with the question. Choose evaluation workflows based on the decision you're trying to make. Are you comparing prompt versions? Testing if the system is good enough to ship? Checking if real users are getting good answers? Let these questions guide your setup – not the metrics alone.

Start small, then grow. Evaluation can feel overwhelming – especially with all the possible metrics, edge cases, and tooling options. It's totally fine to start with a small test set and a couple of key checks and continue with manual review for a while (that's actually recommended since it helps you figure out the criteria). You don't need perfect coverage from day one. Focus on the highest-risk scenarios first and expand from there.

Build collaborative workflows. Make it easy to view, comment on, and review test cases, model outputs, and evaluation scores – especially for non-technical stakeholders. Designing test cases

or giving feedback on LLM system behavior often requires domain knowledge, so it can't just be the engineering team's job. Set up simple interfaces or shared spaces where product owners, support leads, legal, or other reviewers can contribute directly. This applies to both development and production.

Don't forget to keep records – knowing what was tested, when, and how is more valuable than it seems, especially when debugging regressions or making deployment decisions.

Allocate real time and ownership. Evaluation takes effort – writing test cases, reviewing model outputs, maintaining datasets. If it's an afterthought, it won't work. A good idea is to treat it like product infrastructure. Assign owners and make it part of the product release process.

Using Evidently for RAG evaluation

Evidently is an open-source evaluation framework for LLM applications, with over 25 million downloads.

It combines a Python library and a web UI – built to help you run, review, and track the full lifecycle of AI system evaluations, including RAG. You can use it for experiments, regression testing, and ongoing monitoring.

1. Generate synthetic data

You can use Evidently to generate synthetic test cases – tests inputs or question-answer pairs grounded in your actual knowledge base. This helps you bootstrap evaluations for RAG correctness testing.

You can generate data using the Python API or use the UI to inspect and review results. The UI is designed for collaboration – so domain experts can help design or validate test cases and outputs without needing to code.

*You can generate and review synthetic datasets in **Evidently Cloud**.*

2. Run built-in and custom evaluations

Evidently comes with built-in RAG metrics: from ranking metrics like precision@k and hit rate to customizable LLM judges for retrieval and generation quality. These follow the evaluation methods described earlier in this guide.

You can also define your own evaluation criteria and run prompt-based checks for things like correctness, tone, format, or safety using built-in evaluator templates.

If you prefer no-code, you can also run evaluations directly in the UI – just upload your examples, apply LLM judges, and explore results interactively.

3. Track evaluations over time

Evidently is designed for continuous evaluation. You can run tests across experiments and regression checks, or monitor production runs – and track how results evolve over time.

It also includes in-depth tracing – so you can go from a metric drop to specific query failures and dive deep into what went wrong.

Recap

Evaluating RAG systems isn't just about checking if the answer "looks good." It's about understanding how your system performs in real-world conditions – across retrieval, generation, and end-to-end user experience. That means testing more than one thing: search quality, faithfulness, refusals, tone, structure, and more.

Whether you're iterating on prompt formats, experimenting with chunking strategies, or monitoring production reliability, the key is to use evaluation that fits your workflow – and stay focused on what actually fails.

Useful resources:

- Evidently [documentation](#).
 - Theoretical video on [introduction to RAG evaluation](#).
 - RAG evaluation methods: [code tutorial](#).
 - RAG design and evaluation: code [video tutorial](#).
-

WRITTEN BY

SHARE ON

Elena Samuylova

Co-founder and CEO

Evidently AI

Read next

Intro to LLM evals

A gentle introduction to evaluating LLM-powered products. We'll cover the difference between evaluating LLMs and LLM-powered products, evaluation approaches, and how to build the evaluation system.

LLM-as-a-judge

LLM-as-a-judge is a common technique to evaluate LLM-powered products. In this guide, we'll cover how it works, how to build an LLM evaluator and craft good prompts, and what are the alternatives.

© 2026, Evidently AI. All rights reserved
